SudoSLVRR: A Multithreaded Multi-Colony Ant Optimization with Ring and Random Communication Topologies as Sudoku Solver

John Michael E. Jabuen, Gian Myrl Renomeron, and
John Paul T. Yusiong

Division of Natural Science and Mathematics
University of the Philippines Tacloban College
Magsaysay Boulevard, Tacloban City 6500, Leyte, Philippines

Abstract. A popular logic-based combinatorial problem known as Sudoku is computationally expensive because of its interdependent constraints. From traditional solvers to heuristics approaches, although effective when it comes to smaller puzzles, faces difficulties with more complex puzzles. This paper presents SudoSLVRR, a parallel multi-colony ant optimization as sudoku solver that addresses scalability limitations of the existing solvers. This framework combines constraint propagation and multi-colony ant optimization with dynamic collaborative mechanism (DCM-ACO) inside the multithreaded environment. Each multi-colony ant is executed inside each thread where threads exchange their iteration-best solution and best-so-far solution in ring and random communication topologies, respectively. The three-source pheromone update is performed by each thread based on the received information. Experimental results on 9×9, 16×16, and 25×25 Sudoku puzzles shows that the proposed framework was able to achieve higher solution success rate and less solution completion time even in larger Sudoku puzzles when compared to single-colony ant and non-parallel multi-colony ant solvers.

Keywords: Constraint Satisfaction Problem · Swarm Intelligence · Ant Colony Optimization · Dynamic Collaborative Mechanism · Parallel Optimization

1 Introduction

The rule of the logic-based Sudoku game includes filling the grid with numbers from 1 to n without repetition to row, column, and subgrid. A valid Sudoku solution must simultaneously satisfy multiple interdependent constraints which makes it a good benchmark for constraint satisfaction problems [3, 10]. Backtracking is the popular traditional way of solving Sudoku [13, 20, 22]. It tries to solve the puzzle by filling every cell with valid values then goes back to the previous assignment if it leads to invalid puzzle. It was then followed by heuristic approaches [15, 22] who mimicked human reasoning in solving Sudoku. However, these approaches suffered from exponential increase of time. To overcome

this limitation, metaheuristic approaches specifically swarm-based algorithms emerged. Artificial Bee Colony (ABC) [6], Ant Colony Optimization (ACO) [3], and Particle Swarm Optimization (PSO) [9, ?] are a few examples of these approaches. Though ABC's performance showed potential by simulating foraging behaviour, it still struggled on more complex Sudoku puzzles. Unlike PSO and ABC, ACO was able to provide solutions with less execution time thereby indicating that the convergence speed in ACO is faster. Due to its potential, a multi-colony ant optimization was designed to test its performance when there are multiple colonies of ants in the search space. The Dynamic Collaborative Multi-Colony Ant Optimization (DCM-ACO) allows sharing of pheromone information between multiple colonies which improves diversity and convergence speed thus outperforming the single colony [4]. Parallelization of swarm-based algorithms was developed as an effort to improve the performance of the algorithms. One of these is the Parallel Independent Runs (PIR) but no sharing of information between independent agents leads to redundant exploration in the search space [2]. There is another parallelization technique that focuses on ants which leads to synchronization overhead when a single ant updates global pheromone [7].

To address the exponential increase in time and stagnation, this paper proposes SudoSLVRR, a multithreaded sudoku solver that combines constraint propagation and DCM-ACO within threads that communicate according to the communication topologies. SudoSLVRR is designed to balance exploration and exploitation which improves scalability on large-scale Sudoku puzzles.

2 Related Works

3 SudoSLVRR: Algorithm Design and Structure

Algorithm 1: SudoPHASE (Part 1)

```
Input: Sudoku puzzle P
Input: Parameters: τ_0, E(p)*, Con*, timeout
Output: Solved Sudoku puzzle
read puzzle P;
forall cells with fixed values do
    propagate constraints;
                                        // Initial constraint propagation
forall colony do
    initialize pheromone matrix τ_ij ← τ_0;
iter ← 0;
while puzzle is not solved and timeout not reached do
    forall threads in parallel do
        give each ant a local copy of the puzzle;
        assign each ant to a different cell;
        for number of cells do
            for each colony do
                for each ant do
                    if current cell value not fixed then
                        choose value from current cell's value set;
                        fix cell value;
                        propagate constraints;
                        update local pheromone using Equation (2);
                    move to next cell;
        for each colony do
            find the iteration-best in thread using Equation (3);
            calculate the pheromone to add, Δτ, using Equation (4);
            if Δτ iteration-best > global-iteration-best pheromone then
                set global-iteration-best solution to iteration-best solution;
                set Δτ global-iteration-best to Δτ iteration-best;
            if Δτ iteration-best > Δτ best-so-far then
                set best-so-far to iteration-best;
                set Δτ best-so-far to Δτ iteration-best;
            if Δτ best-so-far > Δτ global-best-so-far then
                set global-best-so-far to Δτ best-so-far;
                set global-best-so-far to best-so-far;
        // Apply Dynamic Collaborative Mechanism (DCM-ACO)
        divide ACS colonies into low-entropy and high-entropy groups using
          E(p)*;
        if low-entropy ACS group is not empty then
            apply pheromone fusion with MMAS colony;
        if high-entropy ACS group is not empty then
            allocate and update pheromone using cooperative game theory;
        calculate convergence speed for the MMAS colony;
        if convergence speed < Con* then
            apply public path recommendation from ACS colonies;
        else
            update pheromone for the MMAS colony;
    // Algorithm continues on the next page...
```

Algorithm 1: SudoPHASE (continued)

```
while puzzle is not solved and timeout not reached (continued) do
    // ...continued from previous page
    if communication interval is reached then
        synchronize all threads at barrier;
        generate random matching array;
        exchange iteration-best solutions using ring topology;
        exchange best-so-far solutions using random topology;
        perform global pheromone update using Equation (??);
    else
        perform global pheromone update using Equation (??);
        perform best value evaporation using Equation (??);
    increment iter;
```

3.1 Initial Constraint Propagation

Sudoku has a large solution space and navigating through it requires time. Constraint propagation by its heuristic characteristics narrowed down the solution space. It scans the board and filters out invalid cell-value assignments according to the constraints of Sudoku. Though CP is again called during the solution construction of ants, this preprocessing mechanism allows the ants from each thread to prevent wasting CPU time in trying invalid cell-value assignments.

CP removes invalid assignments using the fixed values (clues) on the puzzle. It implements two rules to systematically prune the solution space. Every cell has its own candidate values and belongs to a specific row, column, and subgrid. Rule 1 collects all fixed values from a certain row, column, and subgrid and removes this set of fixed values from the candidate values of the cell belonging to the same row, column, and subgrid. If there is only one remaining value, CP fixes that value to that cell. The second rule is responsible for fixing hidden singles to a cell. The cell is known to have a hidden single when there is only one remaining value on its candidate values after removing all the candidate values of its peers (cells that reside to the same row, column, and subgrid). This process is recursively done until no further deduction can be made.

By doing this, the puzzle is reduced and the search space is narrowed as the number of fixed cells has increased. However, it is important to note that because of this recursive deduction, there are cells that will be left with no candidate values anymore and it is called "fail cells". The algorithm counts these "fail cells" and will be subtracted from the number of cells filled by the ants to quantify the quality of the constructed solution.

1					7		9	
	3			2				8
		9	6			5		
		5	3			9		
	1			8				2
6					4			
3							1	
	4							7
		7				3		

1	2 5 6 8	2 4 6 8	4 5 8	4 5 3	7	2 4 6	9	3 4 6
4 5 7	3	4 6	1 4 5 9	2	1 5 9	1 4 6 7	4 6 7	8
2 4 7 8	2 7 8	9	6	1 4 3	1 3 8	5	2 3 4 7	1 3 4
2 4 7 8	2 7 8	5	3	1 7 6	1 2 6	9	4 6 7 8	1 4 6
4 7 9	1	4 3	5 7 9	8	5 6 9	4 6 7	3 4 5 6 7	2
6	2 7 8 9	2 3 8	1 2 5 7 9	1 5 7 9	4	1 7 8	3 5 7 8	1 3 5
3	2 5 6 8 9	2 6 8	2 4 5 7 8 9	4 5 6 7 9	2 5 6 8 9	4 2 6 8	1	4 5 6 9
2 5 8 9	4	1	2 5 8 9	3 5 6 9	2 3 5 6 8 9	2 8 6	3 5 6 8	7
2 5 8 9	2 5 6 8 9	7	1 2 4 5 8 9	1 4 5 6 9	1 2 5 6 8 9	3	2 4 5 6 8	4 5 6 9

Fig. 1. AI Escargot Sudoku puzzle (left), and the result after CP (right). The value sets are represented as strings of allowable digit for a certain cell. For example, '2568' represents the set of values 2,5,6,8.

3.2 DCM-ACO

The multi-colony ant optimization for SudoSLVRR is composed of ACS and MMAS colonies. Ant Colony System (ACS) is known for its rapid convergence while Min-Max Ant System has its strong exploratory characteristics. Thus, the two colonies balance the exploitation and exploration mechanism of the algorithm by sharing pheromone information.

Pheromone Matrix Initialization Since the algorithm is composed of multiple colonies, it maintains a pheromone matrix for each colony τij where i represents the index of the cell and j represents the candidate value from the candidate set $\{1, 2, 3, \ldots, d\}$. For a 9×9 Sudoku puzzle, $d = 9$ denotes the dimension of the Sudoku puzzle. All the values in the pheromone matrix are initialized to the same value given by

$$\tau_0 = \frac{1}{c} \tag{1}$$

where $c = d^2$ represents the total number of cells in the puzzle.

Ant Solution Construction The resulting board from the initial constraint propagation is given to each ant to have their own local copy of the board (Line 8). Each ant starts at a random starting cell (Line 9) and moves to the next cell following the left-to-right direction until it goes back to the starting cell (Line 10).

Ant Decision Rule At each visited cell, an ant must choose a value for this cell (Line 14). The greediness of the ant in the ACS colony is determined by the parameter q_0. When a random number $r \in [0, 1]$ is less than q_0, then the ant acts greedily, choosing the value with the highest pheromone concentration from the candidate values of the cell. Otherwise, the ant performs roulette wheel

selection where probabilities of choosing a value is determined by its pheromone concentration. This selection gives the value a chance even with less pheromone concentration to be fixed for the cell. The ants in MMAS colony always perform roulette wheel selection highlighting its exploratory characteristics. Therefore, the q_0 in this colony is set to 0.

Propagate Constraints Propagate Constraints After the ant decides what to fix into the cell (Line 15), it calls the CP to propagate the changes (Line 16). The CP, again, performs the previously discussed rules in removing invalid cell-value assignments. This propagation ensures that every step the ants make is valid while narrowing down the solution space.

Local Pheromone Update To reduce the probability of the certain value of being chosen again, ants perform local pheromone update (Line 17). This update reduces the pheromone concentration of the previously chosen value, encouraging the subsequent ants to explore other values for the cell. The new pheromone concentration is given by:

$$\tau_{ij} \leftarrow (1 - \xi) \cdot \tau_{ij} + \xi \cdot \tau_0 \tag{2}$$

where $\xi = 0.1$.

Solution Evaluation and Iteration-Best Selection After all ants have constructed their solution, the algorithm quantifies the quality of the solution produced by each ant. The iteration-best ant is the ant who has the highest number of filled cells (Line 18). At each iteration, every ant n of the m ants records the total number of cells it was able to fill, f_n. The algorithm compares these scores and sets the f_{best} with the maximum number of filled cells:

$$f_{\text{best}} = \max_{n \in \{1 \ldots m\}} f_n. \tag{3}$$

Pheromone Contribution Calculation The algorithm calculates the pheromone contribution of the iteration-best solution according to

$$\Delta\tau = \frac{c}{c - f_{\text{best}}} \tag{4}$$

The value of the computed $\Delta\tau$ is compared with the current contribution of the best-so-far, $\Delta\tau_{best}$, (with an initial value of 0). If τ is greater than the $\Delta\tau_{best}$, then the algorithm set $\Delta\tau_{\text{best}} \leftarrow \Delta\tau$ Followed by replacing the current best-so-far solution with the iteration-best solution.

4 Results and Discussion

5 Conclusions and Future Work

References

1. M. Dorigo, "Optimization, Learning and Natural Algorithms," Ph.D. Dissertation, Politecnico di Milano, 1992.
2. Q. Yang and L. Fang, "RMACO: A Randomly Matched Parallel Ant Colony Optimization," *World Wide Web*, vol. 19, no. 6, pp. 1009–1022, 2015. [Online]. Available: https://doi.org/10.1007/s11280-015-0373-2
3. H. Lloyd and M. Amos, "Solving Sudoku with Ant Colony Optimization," *IEEE Transactions on Games*, vol. 12, no. 3, pp. 302–313, 2020. [Online]. Available: https://doi.org/10.1109/TG.2020.2969827
4. Y. Mo, Z. Tang, and L. Zhao, "Multi-Colony Ant Optimization with Dynamic Collaborative Mechanism and Cooperative Game," *Information Sciences*, vol. 608, pp. 892–908, 2022. [Online]. Available: https://doi.org/10.1016/j.ins.2022.06.055
5. Y. Manyam, S. Sundaram, and A. Sahay, "Sudoku Solver Algorithms: A Comparative Analysis," in *2024 First International Conference for Women in Computing (InCoWoCo)*, IEEE, 2024, pp. 1–9. [Online]. Available: https://doi.org/10.1109/InCoWoCo58689.2024.10443752
6. J. A. Pacurib, G. M. M. Seno, and J. P. T. Yusiong, "Solving Sudoku Puzzles Using Improved Artificial Bee Colony Algorithm," in *2009 Fourth International Conference on Innovative Computing, Information and Control (ICICIC)*, 2009, pp. 885–888. [Online]. Available: https://doi.org/10.1109/ICICIC.2009.334
7. M. Randall and A. Lewis, "A Parallel Implementation of Ant Colony Optimization," *Journal of Parallel and Distributed Computing*, vol. 62, no. 9, pp. 1421–1432, 2002. [Online]. Available: https://doi.org/10.1006/jpdc.2002.1838
8. P. Malakonakis, M. Smerdis, E. Sotiriades, and A. Dollas, "An FPGA-based Sudoku Solver based on Simulated Annealing methods," in *2009 International Conference on Field-Programmable Technology*, 2009, pp. 522–525. [Online]. Available: https://doi.org/10.1109/FPT.2009.5377608
9. S. McGerty, "Solving Sudoku puzzles with particle swarm optimisation," *Final Report, Macquarie University*, 2009.
10. H. Simonis, "Sudoku as a constraint problem," in *CP Workshop on modeling and reformulating Constraint Satisfaction Problems*, vol. 12, 2005, pp. 13–27.
11. E. C. Chi, "Techniques for Solving Sudoku Puzzles," *arXiv preprint arXiv:1203.2295*, 2012.
12. T. Yato and T. Seta, "Complexity and completeness of finding another solution and its application to puzzles," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. 86, no. 5, pp. 1052–1060, 2003.
13. P. Norvig, "Solving Every Sudoku Puzzle," 2006. [Online]. Available: http://norvig.com/sudoku.html
14. D. Eppstein, "Computational Complexity of Games and Puzzles," in *Proceedings of the International Conference on Fun with Algorithms*, 2006, pp. 1–12.
15. J. F. Crook, "A pencil-and-paper algorithm for solving Sudoku puzzles," *Notices of the AMS*, vol. 56, no. 4, pp. 460–468, 2009.
16. T. Mantere and J. Koljonen, "Solving, rating and generating Sudoku puzzles with GA," in *2007 IEEE Congress on Evolutionary Computation*, IEEE, 2007, pp. 1382–1389.

17. M. Dorigo and T. Stützle, *Ant Colony Optimization*. MIT press, 2004.
18. M. Pedemonte, E. Alba, and F. Luna, "Bit-parallel ACO for the traveling salesman problem," *Applied Intelligence*, vol. 35, no. 3, pp. 346–357, 2011.
19. E.-G. Talbi, *Metaheuristics: From Design to Implementation*. John Wiley & Sons, 2009.
20. M. Schottlender, "The effect of guess choices on the efficiency of a backtracking algorithm in a Sudoku solver," in *IEEE Long Island Systems, Applications and Technology (LISAT) Conference 2014*, 2014, pp. 1–6. [Online]. Available: https://doi.org/10.1109/LISAT.2014.6845190
21. A. Moraglio and J. Togelius, "Geometric particle swarm optimization for the sudoku puzzle," in *Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation*, 2007, pp. 118–125. [Online]. Available: https://doi.org/10.1145/1276958.1276975
22. A. Bhattarai, D. Uprety, P. Pathak, S. N. Shrestha, S. Narkarmi, and S. Sigdel, "A Study Of Sudoku Solving Algorithms: Backtracking and Heuristic," *arXiv preprint arXiv:2507.09708*, 2025.